

Laboratoire 1 — GPIO et délais

Programmation sur STM32F103

Nom : Matisse Rhéaume-Viale

Cours : 243-356-LI - Programmation 3 - Microcontrôleurs et périphériques

Enseignant : Gabriel Fortin-Bélanger **Date :** Mar 8 Septembre

Groupe : 0001

1. Objectif du laboratoire

L'objectif de ce laboratoire est de programmer le microcontrôleur STM32F103 sans utiliser de librairie externe afin de contrôler directement les GPIO.

Le programme doit permettre de :

- Faire clignoter la LED D1-1.
- Contrôler la vitesse de clignotement avec le bouton S1-3.
- Changer la vitesse à chaque appui sur S1-3.
- Faire défiler les périodes suivantes :
 - 300 ms
 - 500 ms
 - 1 s
 - retour à 300 ms

Les fonctions suivantes ont été développées :

- GPIO_InitPin()
- GPIO_WritePin()
- GPIO_ReadPin()
- Systeme_Delai()
- main()

2. Plan de test

2.1 Tableau des tests

Cas de test	Test	Étapes	Résultat attendu	Résultat obtenu
1	Clignotement à vitesse initiale	Mesurer la période du signal sur D1-1 au démarrage.	Période mesurée de 300 ms.	375 ms

Cas de test	Test	Étapes	Résultat attendu	Résultat obtenu
2	Changement de vitesse — 1er appui	Appuyer une fois sur S1-3 et mesurer la période du signal.	Période mesurée de 500 ms.	625 ms
3	Changement de vitesse — 2e appui	Appuyer une deuxième fois sur S1-3 et mesurer la période du signal.	Période mesurée de 1 s.	1250 ms
4	Retour au cycle initial — 3e appui	Appuyer une troisième fois sur S1-3 et mesurer la période du signal.	Période mesurée de 300 ms.	375 ms

2.2 Cas de test 1 — Période initiale de 300 ms

Résultat attendu :

Une période de 300 ms doit être mesurée sur le signal de la LED D1-1.

Résultat mesuré :

[Inscrire la valeur mesurée]

Capture de l'analyseur logique

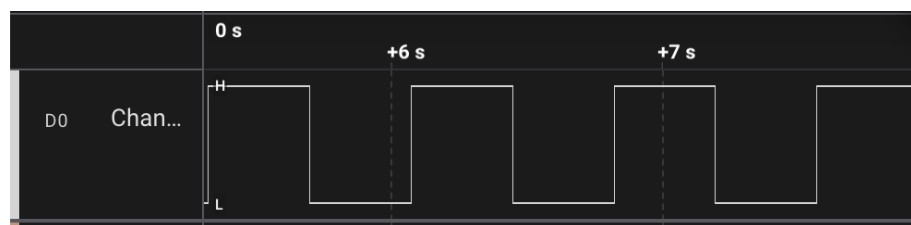


Figure 1: Analyseur logique - 300 ms

Observation :

375 ms de délai donc délai un peu plus grand.

2.3 Cas de test 2 — Période de 500 ms

Après un premier appui sur S1-3, la période doit passer de 300 ms à 500 ms.

Résultat attendu :

500 ms

Résultat mesuré :

625 ms de delai donc delai plus grand

Capture de l'analyseur logique

Figure 2: Analyseur logique - 500 ms

Observation :

Le delai mesuré est different de celui demandé.

2.4 Cas de test 3 — Période de 1 s

Après un deuxième appui sur S1-3, la période doit passer de 500 ms à 1 s.

Résultat attendu :

1 s

Résultat mesuré :

1500 ms

Capture de l'analyseur logique**Observation :**

Le delai est de 1,25s

2.5 Cas de test 4 — Retour à 300 ms

Après un troisième appui sur S1-3, la période doit revenir à 300 ms.

Résultat attendu :

300 ms

Résultat mesuré :

375 ms

Capture de l'analyseur logique**Observation :**

un delai de 375 ms

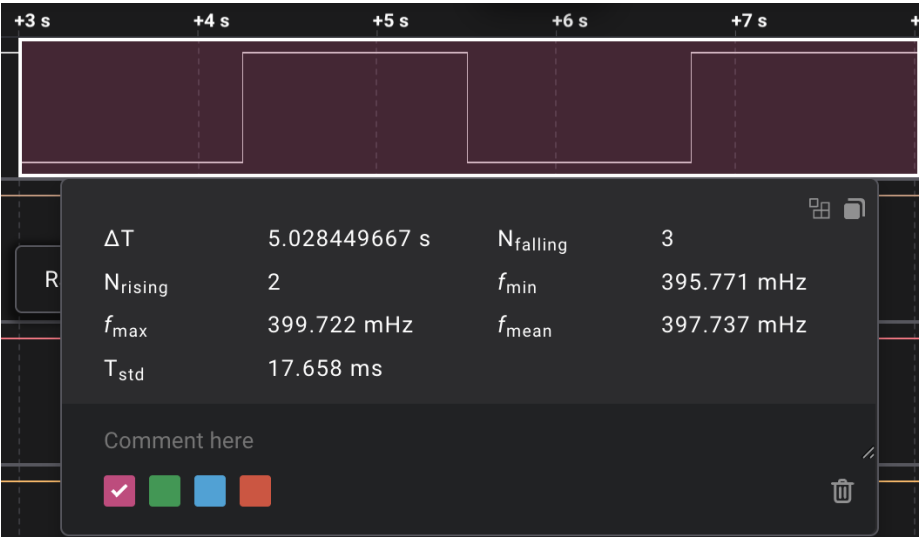


Figure 3: Analyseur logique - 1 seconde

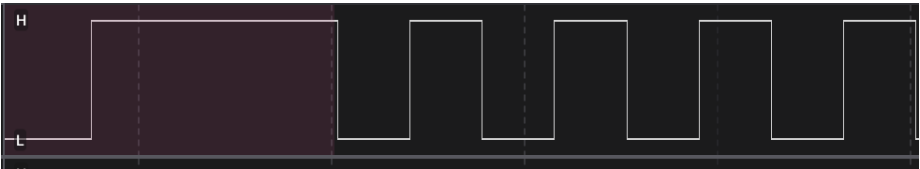


Figure 4: Analyseur logique - retour 300 ms

3. Questions d'analyse

Pour chacune des quatre fonctions demandées ainsi que pour `main()`, les sections suivantes présentent :

1. Une capture du code.
2. Une explication du fonctionnement du code.
3. Une explication des registres utilisés.
4. Une capture de la documentation technique officielle justifiant les configurations.

3.1 Fonction `GPIO_InitPin()`

Capture du code de la fonction

```
void GPIO_InitPin(unsigned char pin, unsigned char mode)
{
    unsigned int decalage;

    RCC_APB2ENR |= (1u << 3);    // Active l'horloge de GPIOB

    decalage = ((unsigned int)pin) * 4u;

    // Efface les 4 bits de configuration de la broche
    GPIOB_CRL &= ~(0xFu << decalage);

    if (mode == 1u)
    {
        // CNF = 00, MODE = 10
        // Sortie push-pull, 2 MHz
        GPIOB_CRL |= (0x2u << decalage);
    }
    else
    {
        // CNF = 01, MODE = 00
        // Entrée flottante
        GPIOB_CRL |= (0x4u << decalage);
    }
}
```

Figure 5: Code `GPIO_InitPin`

Explications du code et des configurations de registres utilisées

Cette fonction configure une broche de GPIOB comme entrée ou comme sortie.

```
RCC_APB2ENR |= (1u << 3);
```

Cette ligne active l'horloge de GPIOB.

```
decalage = ((unsigned int)pin) * 4u;
```

Cette ligne trouve la position des bits de configuration de la broche.

```
GPIOB_CRL &= ~(0xFu << decalage);
```

Cette ligne efface l'ancienne configuration.

Pour une sortie :

```
GPIOB_CRL |= (0x2u << decalage);
```

La broche est configurée en sortie push-pull à 2 MHz.

Pour une entrée :

```
GPIOB_CRL |= (0x4u << decalage);
```

La broche est configurée en entrée flottante.

Registres utilisés

Registre	Adresse	Fonction
RCC_APB2ENR	0x40021018	Active l'horloge de GPIOB.
GPIOB_CRL	0x40010C00	Configure les broches PB0 à PB7.

Justification avec la documentation officielle

9.2.1 Port configuration register low (GPIOx_CRL) (x=A..G)

Address offset: 0x00

Reset value: 0x4444 4444

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CNF7[1:0]		MODE7[1:0]		CNF6[1:0]		MODE6[1:0]		CNF5[1:0]		MODE5[1:0]		CNF4[1:0]		MODE4[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNF3[1:0]		MODE3[1:0]		CNF2[1:0]		MODE2[1:0]		CNF1[1:0]		MODE1[1:0]		CNF0[1:0]		MODE0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Figure 6: Documentation GPIO_InitPin

Explication de la documentation :

La documentation y montre les champs CNF5, MODE5, CNF6, MODE6 et explique les valeurs possibles : entrée, sortie 10 MHz, 2 MHz, 50 MHz, push-pull, pull-up/pull-down, etc.

3.2 Fonction GPIO_WritePin()

Capture du code de la fonction

```
void GPIO_WritePin(unsigned char pin, unsigned char etat)
{
    if (etat == 1u)
    {
        GPIOB_BSRR = (1u << pin);
    }
    else
    {
        GPIOB_BRR = (1u << pin);
    }
}
```

Figure 7: Code GPIO_WritePin

Explication

Cette fonction permet de mettre une broche à 0 ou à 1.

Pour mettre la broche à 1 :

```
GPIOB_BSRR = (1u << pin);
```

Pour mettre la broche à 0 :

```
GPIOB_BRR = (1u << pin);
```

Dans ce laboratoire, PB6 commande la LED D1-1.

Registres utilisés

Registre	Utilité
GPIOB_BSRR	Met la broche choisie à l'état logique 1.
GPIOB_BRR	Met la broche choisie à l'état logique 0.

Justification avec la documentation officielle

Explication de la documentation :

La documentation montre que le registre BSRR permet de mettre une sortie à 1 et que le registre BRR permet de la mettre à 0.

9.2.5 Port bit set/reset register (GPIOx_BSRR) (x=A..G)

Address offset: 0x10

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

Bits 31:16 **BRy**: Port x Reset bit y (y= 0 .. 15)

These bits are write-only and can be accessed in Word mode only.

0: No action on the corresponding ODRx bit

1: Reset the corresponding ODRx bit

Note: If both BSx and BRx are set, BSx has priority.

Bits 15:0 **BSy**: Port x Set bit y (y= 0 .. 15)

These bits are write-only and can be accessed in Word mode only.

0: No action on the corresponding ODRx bit

1: Set the corresponding ODRx bit

Figure 8: Documentation GPIO_WritePin

3.3 Fonction GPIO_ReadPin()

Capture du code de la fonction

```
unsigned char GPIO_ReadPin(unsigned char pin) { if ((GPIOB_IDR & (1u << pin)) != 0u) { return 1u; } else { return 0u; } }
```

Explications du code et des configurations de registres utilisées

Cette fonction lit l'état d'une broche de GPIOB.

`GPIOB_IDR & (1u << pin)`

Cette opération permet de vérifier uniquement le bit de la broche choisie.

Si le bit est à 1, la fonction retourne 1.

Sinon, elle retourne 0.

Dans le laboratoire, cette fonction sert à lire le bouton S1-3 sur PB5.

Registres utilisés

Registre	Utilité
GPIOB_IDR	Permet de lire l'état logique des broches de GPIOB.

Justification avec la documentation officielle

9.2.3 Port input data register (GPIOx_IDR) (x=A..G)

Address offset: 0x08h

Reset value: 0x0000 XXXX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IDR15	IDR14	IDR13	IDR12	IDR11	IDR10	IDR9	IDR8	IDR7	IDR6	IDR5	IDR4	IDR3	IDR2	IDR1	IDR0
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **IDRy**: Port input data (y= 0 .. 15)

These bits are read only and can be accessed in Word mode only. They contain the input value of the corresponding I/O port.

Figure 9: Documentation GPIO_ReadPin

Explication de la documentation :

La documentation montre que le registre IDR contient l'état logique des broches du port GPIO.

3.4 Fonction Systeme_Delai()

Capture du code de la fonction

```
void Systeme_Delai(unsigned int delai) { volatile unsigned int delaiOut = 0u;
volatile unsigned int delaiIn;

while (delaiOut < delai)
{
    delaiOut++;

    delaiIn = 0u;

    while (delaiIn < 1000u)
    {
        delaiIn++;
    }
}
}
```

Explications du code et des configurations utilisées

Cette fonction crée un délai logiciel à l'aide de deux boucles.

Le paramètre `delai` indique la durée désirée.

La boucle externe est répétée selon la valeur de `delai`, tandis que la boucle interne ralentit l'exécution du programme.

Les variables sont déclarées `volatile` afin que le compilateur n'enlève pas les boucles.

Le délai est approximatif et dépend de l'horloge HSI de 8 MHz.

Configuration utilisée

Élément	Valeur / rôle
Horloge du processeur	HSI 8 MHz
Paramètre <code>delai</code>	Indique la durée désirée
Boucle interne	Crée le délai logiciel
<code>volatile</code>	Empêche l'optimisation des compteurs

Justification avec la documentation officielle

7.2.2 HSI clock

The HSI clock signal is generated from an internal 8 MHz RC Oscillator and can be used directly as a system clock or divided by 2 to be used as PLL input.

The HSI RC oscillator has the advantage of providing a clock source at low cost (no external components). It also has a faster startup time than the HSE crystal oscillator however, even with calibration the frequency is less accurate than an external crystal oscillator or ceramic resonator.

Figure 10: Documentation horloge HSI

Explication de la documentation :

La documentation indique que l'oscillateur interne HSI fonctionne à 8 MHz. Cette horloge influence la vitesse d'exécution des boucles utilisées pour créer le délai.

3.5 Fonction `main()`

Capture du code de la fonction

```
int main(void) { unsigned int delai = 300u;

unsigned char boutonActuel;
unsigned char boutonPrecedent = 1u;

GPIO_InitPin(6u, 1u); // PB6 comme LED D1-1

GPIO_InitPin(7u, 0u); // PB7 comme bouton S1-3
```

```

GPIO_WritePin(6u, 1u); // LED d1-1 éteinte

while (1)
{
    boutonActuel = GPIO_ReadPin(7u); // Lire l'état du bouton S1-3

    /* -----
    * Function : Systeme_Delai
    * Author   : Matisse Rhéaume-Viale & PROG3
    * Desc     : Produit un délai logiciel approximatif en millisecondes.
    * Notes    : Boucle calibrée pour l'horloge HSI de 8 MHz.
    * ----- */
    if ((boutonPrecedent == 1u) && (boutonActuel == 0u)) // Détection du front descendant
    { Systeme_Delai(20u); // Délai anti-rebond de 20 ms
    if (GPIO_ReadPin(7u) == 0u) // Vérifie que le bouton est toujours appuyé
    {
        if (delai == 300u) // 300 ms -> 500 ms
        {
            delai = 500u;
        }
        else if (delai == 500u) // 500 ms -> 1000 ms
        {
            delai = 1000u;
        }
        else // 1000 ms -> 300 ms
        {
            delai = 300u;
        }
    }
    }

    boutonPrecedent = boutonActuel;

    GPIO_WritePin(6u, 0u); // Allumer D1-1

    Systeme_Delai(delai); // Délai entre l'allumage et l'extinction de la LED

    GPIO_WritePin(6u, 1u); // Éteindre D1-1

    Systeme_Delai(delai); // Délai entre l'extinction et l'allumage de la LED
    }
}

```

Explications du code et des configurations utilisées

La fonction `main()` contrôle le fonctionnement général du programme.

PB6 est configurée comme sortie pour la LED :

```
GPIO_InitPin(6u, 1u);
```

PB5 est configurée comme entrée pour le bouton :

```
GPIO_InitPin(5u, 0u);
```

Au démarrage, la vitesse est :

```
delai = 300u;
```

Le programme détecte un appui sur le bouton avec :

```
if ((boutonPrecedent == 1u) && (boutonActuel == 0u))
```

Un délai de 20 ms est utilisé pour réduire le rebond du bouton.

À chaque appui, la vitesse change :

300 ms → 500 ms → 1000 ms → 300 ms

La LED est ensuite allumée et éteinte avec `GPIO_WritePin()` et `Système_Delai()`.

Initialisation

- PB6 est configurée comme sortie pour commander D1-1.
- PB5 est configurée comme entrée pour lire S1-3.
- La vitesse initiale est réglée à 300 ms.

Gestion du bouton

Le programme détecte un appui sur S1-3 en observant le changement d'état du bouton.

Un délai anti-rebond est appliqué avant de confirmer l'appui.

À chaque appui valide, la valeur du délai suit la séquence :

“text 300 ms → 500 ms → 1000 ms → 300 ms → ... Gestion de la LED

La LED est alternativement allumée et éteinte à l'aide de `GPIO_WritePin()`.

La fonction `Système_Delai()` contrôle la durée du clignotement.

Justification avec la documentation officielle

Explication : La fonction `main()` contrôle le fonctionnement général du programme.

PB6 est configurée comme sortie pour commander la LED D1-1 et PB5 comme entrée pour lire le bouton S1-3.

La vitesse initiale est de 300 ms.

À chaque appui sur S1-3, le délai change selon la séquence :

“text 300 ms $\rightarrow$ 500 ms $\rightarrow$ 1000 ms $\rightarrow$ 300 ms

Un délai de 20 ms est utilisé comme anti-rebond du bouton.

La LED est ensuite allumée et éteinte avec `GPIO_WritePin()` et les délais sont produits avec `Systeme_Delai()`.

Justification avec la documentation officielle

Les configurations utilisées dans `main()` sont justifiées par les sections précédentes sur les registres GPIO.

4. Conclusion

Les quatre cas de test ont permis de vérifier le fonctionnement du programme.

Les périodes mesurées sont de 375 ms, 625 ms et 1250 ms au lieu des valeurs attendues de 300 ms, 500 ms et 1000 ms.

Les périodes mesurées sont donc environ 25 % plus grandes que les valeurs attendues. Cet écart vient de la fonction `Systeme_Delai()`, qui utilise des boucles logicielles et produit donc un délai approximatif.

Le bouton S1-3 permet tout de même de changer correctement entre les trois vitesses et de revenir à la vitesse initiale.